SpeXFeed MVP 1.1 — ROD Name Registration and Profile Binding Plan

1. Executive summary

SpeXFeed MVP 1 should start as a SpaceXpanse/OpenSpeX Nostr client that uses the ROD blockchain only for relay discovery. MVP 1.1 should add the next most useful blockchain primitive: **registering ROD names and connecting them to Nostr profiles**.

This is not SpeXID yet.

The correct framing is:

> **SpeXFeed names are simple ROD-backed profile handles that can point to a Nostr public key and basic profile metadata.**

The purpose is not full identity infrastructure. The purpose is to give users a human-readable, blockchain-anchored profile name inside SpeXFeed.

In practical terms:

```
Nostr profile = social identity
ROD name = blockchain-owned handle / anchor
SpeXFeed = UI that connects them
```

This gives SpeXFeed a useful native feature without overbuilding the future identity layer.

2. Why this should be added

The relay discovery MVP solves:

> "Where is the SpaceXpanse/OpenSpeX Nostr network?"

Name registration solves:

> "Who is this profile, and is this human-readable name actually controlled by the same person?"

Nostr public keys are powerful but unreadable. A public key like `npub1...` is not suitable for mainstream users, community coordination, or future reputation systems. A ROD-backed name gives users a recognizable handle that is harder to impersonate if the client verifies it correctly.

This also prepares the ground for future OpenSpeX development:

```
simple name binding
↓
profile verification
↓
proof posts
↓
reputation
↓
SpeXID
↓
OpenSpeX agent/user/service identity
```

But MVP 1.1 must stop at simple name binding.

3. What this feature is

SpeXFeed MVP 1.1 adds:

1. Search for available ROD names.
2. Register a new ROD name.
3. Attach a Nostr public key to that ROD name.
4. Display verified ROD names on SpeXFeed profiles.
5. Allow users to update the ROD name record.
6. Allow users to remove or rotate the linked Nostr key.
7. Allow users to browse a profile by name.
8. Keep all existing Blockcore Notes/Nostr profile features working.

The user-facing result:

```
@alice
alice.rod / alice.spex-style display
Linked to npub1...
Verified by ROD name record
```

The exact public suffix can be decided later. Internally, the blockchain record should use ROD namespaces.

4. What this feature is not

SpeXFeed MVP 1.1 is not:

```
not SpeXID
not KYC
not legal identity
not a wallet replacement
not universal login
not reputation
```

```
not proof-of-personhood
not OpenID/OAuth
not agent passport
not a full decentralized DNS resolver
```

Do not call it SpeXID yet. That name should be reserved for the stronger identity layer later.

The safest product name for this feature is:

> **ROD Profile Name**

or:

> **SpeXFeed Name**

Recommended:

> **SpeXFeed Name powered by ROD**

5. Recommended namespace

Use a dedicated namespace for SpeXFeed profile names.

Best option:

```
sf/<name>
```

Examples:

```
sf/alice
sf/darma
sf/voidrunner
sf/bello
```

Why `sf/` is good:

It is short, fits the ROD name model, and keeps SpeXFeed profile names separate from future identity records, app records, relay records, game records, and service records.

Avoid using `id/` for now because that smells like SpeXID. Avoid `d/` because that should be reserved for DNS-style naming. Avoid `app/` because that is already better for app discovery records such as `app/spexfeed`.

Recommended namespace split:

```
app/spexfeed          = SpeXFeed app relay discovery
relay/spexfeed.main   = optional future relay-set record
sf/alice              = SpeXFeed profile name
id/alice              = future SpeXID identity, not MVP 1.1
svc/provider          = future service provider record
```

6. ROD name record schema

The name value must stay compact because ROD name values are small. The current SpaceXpanse documentation describes ROD names/accounts as blockchain records whose owner can update attached values, with values usable for decentralized app/game data. The broader ROD docs describe the blockchain as supporting arbitrary names, digital IDs, and secure value/data storage. Because of this, the profile record must be compact, predictable, and versioned.

Recommended schema:

```
{
  "v": 1,
  "t": "spexfeed.profile",
  "npub": "npub1...",
  "pub": "hex_nostr_pubkey",
  "name": "alice",
  "display": "Alice",
  "about": "Builder in SpaceXpanse/OpenSpeX",
  "picture": "https://example.com/avatar.png",
  "relays": [
    "wss://relay.openspex.org",
    "wss://relay.spacexpanse.org"
  ],
  "updated": 1764979200
}
```

A more compact production form:

```
{
  "v":1,
  "t":"sf.profile",
  "p":"hex_nostr_pubkey",
  "n":"alice",
  "d":"Alice",
  "a":"Builder in SpaceXpanse/OpenSpeX",
  "i":"https://example.com/avatar.png",
  "r":["wss://relay.openspex.org"],
  "u":1764979200
}
```

Recommended compact fields:

```
v = schema version
t = record type
p = Nostr public key in hex
n = handle/name
d = display name
a = about/bio
i = image/avatar URL
r = preferred relays
u = updated timestamp
```

For MVP 1.1, the only required fields should be:

```
v
t
p
n
u
```

Everything else is optional.

7. Why store the Nostr public key in hex

Nostr uses hex public keys internally. `npub` is user-friendly, but it is an encoded form. The ROD name record should store the canonical hex public key.

The UI can display:

```
npub1...
```

but the record should store:

```
p: "hex_nostr_pubkey"
```

This avoids ambiguity and makes verification easier.

8. Profile binding model

There are two possible binding models.

Model A — ROD-only binding

The ROD record says:

```
sf/alice → nostr public key
```

This is simple and enough for MVP 1.1.

Model B — Mutual binding

The ROD record points to the Nostr public key, and the Nostr profile also points back to the ROD name.

Example Nostr profile metadata includes:

```
{
  "name": "alice",
  "display_name": "Alice",
  "about": "Builder in SpaceXpanse/OpenSpeX",
  "picture": "https://example.com/avatar.png",
  "spexfeed_name": "sf/alice"
}
```

Recommended MVP 1.1 approach:

> Use **mutual binding**, but treat the ROD record as the source of truth.

Verification rule:

```
Strong verified:
ROD record points to Nostr public key
AND Nostr profile metadata points back to the ROD name

Basic verified:
ROD record points to Nostr public key

Unverified claim:
Nostr profile says it owns a ROD name, but ROD record does not point back
```

UI labels:

```
Verified ROD name
Partially linked
Unverified claim
Mismatch warning
```

9. User flow: registering a name

Flow

```
User opens SpeXFeed
↓
Connects Nostr signer
↓
Opens “Register SpeXFeed Name”
↓
Searches name availability
↓
Chooses name
↓
SpeXFeed prepares ROD name registration
↓
User confirms in ROD wallet / local daemon / helper wallet
↓
Transaction broadcasts
↓
SpeXFeed waits for confirmation
↓
User sees “Name pending”
↓
After confirmation, profile shows verified name
```

Important UX detail

The app should not pretend registration is instant.

Use states:

```
Available
Unavailable
Invalid
Pending registration
Registered, waiting for confirmation
Registered and linked
Registered but linked to another key
Owned but not linked
```

10. Name rules

Use strict names from day one.

Recommended rules:

```
Length: 3–32 characters
Allowed: lowercase a-z, numbers 0-9, hyphen
Must start with letter or number
Must end with letter or number
No spaces
No uppercase
No emoji
No slash inside chosen handle
No reserved names
```

Examples:

```
valid:
alice
bello
void-runner
agent007

invalid:
Alice
al
open/spex
-spex
spex-
space xpanse
```

Reserved names:

```
admin
root
support
spex
openspex
spacexpanse
rod
metalog
launchpad
voidrunner
orbitchallenge
darma
relay
wallet
api
official
foundation
```

Reason: do not allow users to grab names that look official.

11. Registration methods

There are three implementation levels.

Method 1 — Local ROD daemon RPC

SpeXFeed calls a local or backend service that talks to `spacexpansed`.

Good for internal MVP/testing.

Pros:

```
fast to implement
uses existing ROD infrastructure
clear ownership model
```

Cons:

```
not mainstream-friendly
requires local wallet/node or trusted backend
```

Method 2 — Companion registration gateway

A SpeXFeed backend prepares transactions or calls a controlled wallet service for early users.

Pros:

```
easier onboarding
good for demo
```

Cons:

```
more centralized
custody/security risks if not carefully designed
```

Method 3 — Browser wallet / Coinb.in-style flow

The app generates or uses a ROD address, builds a transaction, and lets the user sign/broadcast.

Pros:

```
future-friendly
user-controlled
fits web app
```

Cons:

```
more implementation complexity
riskier if rushed
```

Recommended MVP 1.1:

> Start with **Method 1 for internal/dev** and **Method 2 for controlled public alpha**. Do not implement browser custody until the ROD wallet flow is designed properly.

The minimum public version can say:

> “Name registration currently requires ROD wallet connection / helper service. SpeXFeed does not store your ROD private keys.”

12. Backend API

SpeXFeed should not expose raw wallet RPC credentials in the browser.

Use a backend API.

Minimal endpoints:

```
GET  /api/rod/name/:name
GET  /api/rod/name/:name/status
POST /api/rod/name/prepare-register
POST /api/rod/name/prepare-update
POST /api/rod/tx/broadcast
GET  /api/rod/tx/:txid
```

For controlled alpha, if the backend handles registration:

```
POST /api/spexfeed/names/register-request
POST /api/spexfeed/names/update-request
GET  /api/spexfeed/names/request/:id
```

But avoid custody if possible.

13. Name availability check

Input:

```
alice
```

Canonical ROD name:

```
sf/alice
```

Availability process:

```
Normalize input
↓
Validate name rules
↓
Convert to sf/<name>
↓
Query ROD name
↓
Return status
```

Possible API response:

```
{
  "input": "alice",
  "rodName": "sf/alice",
  "valid": true,
  "available": false,
  "ownerKnown": true,
  "recordType": "sf.profile",
  "linkedPubkey": "hex_nostr_pubkey",
  "display": "Alice"
}
```

14. Profile verification process

When viewing a Nostr profile:

```
1. Load Nostr profile metadata.
2. Check whether profile declares a SpeXFeed name.
3. If yes, query ROD record.
4. Verify record type.
5. Compare record public key with viewed profile public key.
6. Show status.
```

When viewing a ROD name:

```
1. Query ROD record sf/alice.
2. Extract linked Nostr public key.
3. Load Nostr profile from relays.
4. Show profile page.
5. Show binding status.
```

15. UI components

A. Name badge

Profile header:

```
Alice
@alice
Verified SpeXFeed Name: sf/alice
Nostr: npub1...
```

Badge states:

```
ROD verified
ROD pending
ROD mismatch
ROD name not found
ROD lookup unavailable
```

B. Register name page

Route:

```
/settings/name
/register-name
```

Fields:

```
Choose name
Display preview
Nostr public key
Optional display name
Optional bio
Optional avatar
Preferred relay
```

Buttons:

```
Check availability
Register name
Update linked profile
Refresh status
```

C. Name search

Search bar:

```
Search SpeXFeed name or Nostr npub
```

Input examples:

```
alice
sf/alice
npub1...
```

Behavior:

```
If normal text → check sf/<text>
If sf/name → query exact ROD name
If npub → open Nostr profile
```

D. Profile settings

Add section:

```
SpeXFeed Name

Current Nostr public key:
npub1...

Linked ROD name:
sf/alice

Status:
Verified

Actions:
Update record
Remove link
Register another name
```

16. Record update logic

Users should be able to update:

```
display name
about/bio
avatar
preferred relays
linked Nostr public key
```

But changing the linked public key is sensitive. It should show a warning:

> “Changing the linked Nostr key will move this name to another profile. People following your old profile may see a mismatch until they refresh.”

Recommended update flow:

```
Load current record
↓
Edit fields
↓
Preview JSON
↓
Validate size
↓
Prepare ROD name update
↓
Sign/broadcast
↓
Wait for confirmation
```

17. Impersonation handling

This is crucial.

A Nostr user can write in their metadata:

```
spexfeed_name: sf/alice
```

That claim must not be trusted unless the ROD record confirms it.

UI rules:

```
If Nostr claims name but ROD does not point back:
Show “Unverified claim”
```

```
If ROD points to different pubkey:
Show “Name mismatch”

If ROD points to current pubkey:
Show “Verified”

If ROD lookup fails:
Show “Unable to verify”
```

Never display an unverified claimed name as if it were verified.

18. Privacy tradeoff

Linking a ROD name to a Nostr public key is public.

SpeXFeed must tell the user:

```
This creates a public link between your blockchain name and your Nostr
profile.
Anyone can see this connection.
Do not use this if you want those identities separated.
```

This warning must appear before registration.

19. Why this is not SpeXID yet

SpeXID should eventually support broader identity behavior:

```
login/authentication
claims
service access
payments
reputation
agent profiles
OpenID-like flows
cross-app identity
```

MVP 1.1 does not need that.

SpeXFeed names should be described as:

> **public profile handles anchored in ROD**

not:

> **full digital identities**

This avoids overclaiming.

20. Integration with relay discovery

Keep relay discovery and profile names separate.

Relay discovery record:

```
app/spexfeed
```

Profile name record:

```
sf/alice
```

They should use different schemas:

```
app/spexfeed → spexfeed.relays
sf/alice → sf.profile
```

This makes the system modular.

Future client startup:

```
Load app/spexfeed
↓
Connect to canonical relays
↓
Load Nostr profile
↓
If profile declares sf/name, verify via ROD
```

21. Future compatibility with OpenSpeX

This design prepares future features without requiring them now.

Later, OpenSpeX can add:

```
proof posts linked to sf/name
reputation summaries linked to sf/name
task history linked to sf/name
validator roles linked to sf/name
agent profiles linked to sf/name
```

```
ROD payment address linked to sf/name
SpeXID upgrade path from sf/name to id/name
```

Possible migration:

```
sf/alice
↓
id/alice
```

or:

```
sf/alice remains social handle
id/alice becomes full SpeXID identity
```

Recommended future distinction:

```
sf/name = SpeXFeed social profile
id/name = SpeXID identity
svc/name = service/provider
agent/name = AI agent identity
```

22. MVP 1.1 implementation sequence

Sprint 1 — Schema and validation

Deliverables:

```
sf.profile schema v1
name rules
reserved-name list
record validator
sample records
size checker
```

Acceptance criteria:

```
Valid profile record passes.
Oversized record fails.
Invalid relay URL fails.
Invalid name fails.
Missing public key fails.
Malformed JSON fails.
```

Sprint 2 — ROD name lookup

Deliverables:

```
GET /api/rod/name/:name
availability check
record parser
profile resolver
```

Acceptance criteria:

```
Can check if sf/alice exists.
Can parse sf.profile record.
Can return linked Nostr pubkey.
Can identify invalid or wrong-type records.
```

Sprint 3 — Profile badge

Deliverables:

```
profile badge component
verification state machine
ROD lookup on profile view
settings display
```

Acceptance criteria:

```
Verified name displays correctly.
Mismatch shows warning.
Unavailable lookup does not break profile page.
Unverified claims are clearly marked.
```

Sprint 4 — Name registration flow

Deliverables:

```
register-name page
availability check
registration request builder
update request builder
confirmation tracking
```

Acceptance criteria:

```
User can check name.
User can register available name through supported ROD flow.
User sees pending state.
User sees verified state after confirmation.
```

Sprint 5 — Profile update flow

Deliverables:

```
edit ROD profile record
preview JSON
validate size
submit update
track confirmation
```

Acceptance criteria:

```
User can update display metadata.
User can update preferred relay.
User can rotate linked Nostr key with warning.
Client detects updated record.
```

Sprint 6 — Documentation and alpha release

Deliverables:

```
ROD-NAME-PROFILE-BINDING.md
SF-PROFILE-SCHEMA-v1.md
NAME-REGISTRATION-FLOW.md
SECURITY-AND-PRIVACY.md
MVP-1.1-README.md
```

Acceptance criteria:

```
A new tester can understand how ROD names connect to Nostr profiles.
The docs clearly say this is not SpeXID.
The docs explain privacy risks.
The docs explain verification states.
```

23. Developer data structures

TypeScript interface

```ts
export interface SpeXFeedProfileRecordV1 {
  v: 1;
  t: 'sf.profile';
  p: string;        // hex nostr pubkey
  n: string;        // handle without sf/
  d?: string;       // display name
  a?: string;       // about
  i?: string;       // image/avatar URL
  r?: string[];     // preferred relays
  u: number;        // unix updated timestamp
}
```

Verification result

```ts
export type SpeXFeedNameStatus =
  | 'verified'
  | 'partial'
  | 'unverified_claim'
  | 'mismatch'
  | 'not_found'
  | 'pending'
  | 'lookup_failed'
  | 'invalid_record';

export interface SpeXFeedNameVerification {
  status: SpeXFeedNameStatus;
  rodName: string;
  handle: string;
  expectedPubkey?: string;
  actualPubkey: string;
  record?: SpeXFeedProfileRecordV1;
  confirmations?: number;
  txid?: string;
  warning?: string;
}
```

24. Minimal backend response examples

Available name

```json
{
  "rodName": "sf/alice",
  "handle": "alice",
```

```
  "valid": true,
  "available": true
}
```

Registered name

```
{
  "rodName": "sf/alice",
  "handle": "alice",
  "valid": true,
  "available": false,
  "record": {
    "v": 1,
    "t": "sf.profile",
    "p": "hex_nostr_pubkey",
    "n": "alice",
    "d": "Alice",
    "u": 1764979200
  },
  "confirmations": 120
}
```

Invalid name

```
{
  "rodName": null,
  "handle": "sp ace",
  "valid": false,
  "available": false,
  "error": "Only lowercase letters, numbers, and hyphens are allowed."
}
```

25. Recommended MVP copy

Register page intro

> Register a SpeXFeed Name on the ROD blockchain and link it to your Nostr profile. This gives your profile a human-readable, blockchain-anchored handle inside SpeXFeed.

Privacy warning

> This creates a public link between your ROD name and your Nostr public key. Anyone can verify this connection. Do not register if you want to keep these identities separate.

Not SpeXID warning

> SpeXFeed Names are simple profile handles. They are not SpeXID, legal identity, KYC, or proof-of-personhood.

Verified badge tooltip

> This profile is linked to a ROD blockchain name record that points to this Nostr public key.

26. Strategic verdict

Adding name registration is worth doing, but only if it stays narrow.

The correct MVP 1.1 is:

```
Register sf/name on ROD
↓
Store linked Nostr public key in the name value
↓
Verify the binding inside SpeXFeed profiles
↓
Display a human-readable blockchain-backed profile handle
```

This gives SpeXFeed a native SpaceXpanse feature without dragging in the full SpeXID stack.

The clean architecture becomes:

```
app/spexfeed
= ROD record for relay discovery

sf/alice
= ROD record for SpeXFeed profile binding

Nostr profile
= social profile and posting identity

SpeXFeed
= UI that connects ROD discovery + ROD profile names + Nostr feeds
```

The most important design rule:

> **ROD owns the name. Nostr carries the conversation. SpeXFeed verifies the connection.**

That is the right bridge between SpaceXpanse blockchain infrastructure and Nostr social infrastructure.